Semantic Alignment and Risk Prediction in Software Pull Requests using SBERT and Random Forest

Abstract

In the evolving landscape of collaborative software development, particularly in environments that rely heavily on distributed teams and asynchronous workflows, ensuring that implemented code matches the intention of assigned tasks is critical. This paper explores a novel, data-driven approach that enhances semantic traceability and operational quality control in the software lifecycle. By combining Sentence-BERT (SBERT) for semantic similarity measurement with a Random Forest classifier for risk prediction, our solution intelligently flags misaligned and high-risk pull requests (PRs). This research is grounded in a synthetic dataset that mimics real-world software engineering scenarios involving JIRA tickets and Git-based code submissions. The proposed model offers interpretable, scalable insights that can significantly improve code review efficiency, accuracy, and overall software quality.

1. Introduction

Software engineering has matured into a highly collaborative and iterative discipline. Project management tools like JIRA are often used in tandem with version control platforms such as GitHub or GitLab. Developers are assigned tasks as tickets, and the resolution of these tasks is usually submitted as pull requests for review and merging. However, the assumption that each PR fully and accurately addresses the scope of its corresponding JIRA ticket often fails under practical scrutiny.

Such discrepancies may not only introduce bugs but also reduce the long-term maintainability of the codebase. Manual reviews, while helpful, are time-consuming and subject to cognitive biases. In large organizations with high volumes of PRs, reviewers are forced to prioritize based on heuristics and familiarity. This increases the risk that semantically misaligned or technically risky PRs may be merged undetected.

This research proposes an automated framework that acts as a first layer of defense in the review pipeline. It does this by evaluating the semantic similarity between ticket and PR descriptions and using engineered features to predict risk with the help of a supervised machine learning model.

2. Data Generation and Simulation

To facilitate controlled experimentation, a synthetic dataset was generated using a simulation script. The script creates 500 JIRA tickets and 500 corresponding PRs that reflect realistic development activities, including feature additions, bug fixes, and performance improvements. Descriptions are populated with diverse, domain-relevant templates and technology stacks. Metadata such as priority, file changes, lines added/deleted, and comment count are also included.

In order to mimic real-world inconsistencies, approximately 10% of the PRs were intentionally misaligned from their respective JIRA tickets by modifying descriptions or misattributing the ticket ID. This provides a necessary class of negative samples for training and evaluation of the alignment model.

3. Semantic Similarity Modeling Using SBERT

SBERT (Sentence-BERT) is a modification of the BERT architecture specifically designed for generating semantically meaningful sentence embeddings. Traditional BERT models are not suitable for

computing cosine similarity directly between sentences because their output is not optimized for sentence-level representations. SBERT addresses this by applying a Siamese or triplet network structure and fine-tuning it using Natural Language Inference (NLI) datasets.

In this work, we use the pre-trained all-MiniLM-L6-v2 variant of SBERT, which balances performance and speed. For each JIRA-PR pair, we extract the respective descriptions and pass them through SBERT to obtain dense vector embeddings. These embeddings are then compared using cosine similarity. A high similarity score suggests that the PR description is semantically aligned with the ticket it purports to resolve, while a low score indicates potential deviation.

The use of SBERT provides several advantages. It captures contextual meaning rather than relying on keyword overlap. This is especially important in technical domains where synonyms, abbreviations, or paraphrasing are common. For example, "fix null response" and "resolve empty API output" should yield high similarity despite their lexical differences.

4. Alignment Scoring and Thresholding

The output of the SBERT model for each pair is a floating-point score between -1 and 1, although in practice values range between 0 and 1 due to the nature of cosine similarity. We set a threshold of 0.7, below which the pair is flagged for potential misalignment. This threshold was selected based on empirical observation and can be tuned in production based on organizational tolerance for false positives versus false negatives.

Once the scores are calculated, they are saved in a dataset that includes PR IDs, descriptions, and alignment scores. The dataset also retains the original labels to support downstream risk modeling. The SBERT stage, therefore, serves as both a feature generator and a diagnostic tool for semantic coherence.

5. Risk Labeling and Feature Engineering

A key contribution of this work is the creation of a hybrid labeling strategy for identifying high-risk pull requests. Instead of relying solely on historical data or reviewer annotations—which are often unavailable—we define a rule-based schema to generate pseudo-ground truth labels.

A PR is considered high-risk if it satisfies two or more of the following:

- It has a low semantic similarity score (< 0.7),
- It modifies a large number of lines (more than 120 additions or 80 deletions),
- It changes multiple critical files (e.g., related to authentication or database logic),
- It lacks peer review comments,
- It touches high-risk areas in the codebase.

This logic mimics the intuition of experienced reviewers and allows the machine learning model to learn from a balanced and explainable dataset. In total, each PR is characterized using features such as:

- Number of critical files touched,
- Lines added,
- Lines deleted,
- Total lines modified,
- Author experience (derived from historical contribution counts),
- Average risk score per file,
- SBERT semantic similarity score,
- Code churn ratio,
- Number of review comments.

6. Predictive Modeling with Random Forest

With labelled data and engineered features in hand, we use a Random Forest classifier to model the probability of a PR being high-risk. Random Forest is chosen for its robustness, interpretability, and ability to handle both linear and nonlinear feature interactions without requiring normalization or extensive preprocessing.

We perform an 80/20 train-test split with stratified sampling to ensure class balance. The Random Forest model is configured with 100 trees and a maximum depth of 5 to prevent overfitting. During training, the model learns to associate feature combinations with the high-risk label using ensemble decision trees that aggregate their results.

Evaluation metrics include accuracy, precision, recall, and F1-score. The model achieves:

- Accuracy: ~0.91
- Precision: ~0.89
- Recall: ~0.87
- F1-score: ~0.88

The most important features, based on Gini importance, are:

- Semantic similarity score from SBERT,
- Lines added,
- Critical file count,
- Number of comments.

These results confirm that semantic alignment is a strong indicator of risk and that integrating NLP-based features with structural code metrics can lead to a powerful composite model.

7. Model Deployment and Output

Once trained and validated, the model is used to classify the full dataset. Each PR is assigned a predicted label ("High Risk" or "Low Risk") along with a probability score indicating the confidence level. The results are stored in a predictions file for integration with dashboards, CI/CD pipelines, or alerting systems.

This enables real-time feedback during code submission and can act as a gate in the pull request lifecycle. High-risk PRs can be routed for deeper manual review, while low-risk, well-aligned PRs can be fast-tracked—thus optimizing reviewer effort.

8. Real-World Applications

In an enterprise setting, this system could be deployed as a service that listens to new PRs, fetches the corresponding JIRA tickets, computes similarity scores, and invokes the classifier to flag issues. Reviewers can see risk scores directly in the interface, with the option to expand details explaining why a PR was flagged.

For project managers, the system offers insights into alignment trends over time, team-level consistency, and review backlog prioritization. For developers, it acts as a coaching tool, helping them write better-aligned descriptions and reducing the likelihood of their PRs being flagged.

9. Limitations and Challenges

While powerful, the system is not without limitations. Semantic similarity models may struggle with highly technical or abbreviated language. Training data is synthetic, so real-world generalizability must be validated. Also, the binary risk labeling is rule-based and could benefit from human-in-the-loop feedback to evolve dynamically.

The model also assumes that PR descriptions are available and meaningful—something that varies significantly in practice. Integrating static code analysis and diff analysis could further enhance predictive performance.

10. Conclusion

This study demonstrates the effectiveness of combining SBERT's semantic understanding with Random Forest's structured decision-making capabilities to address a longstanding challenge in software engineering: ensuring that PRs align with assigned tasks and do not pose undue risk.

By automating the detection of semantic misalignment and predicting PR risk, this system empowers developers, reviewers, and managers to maintain code quality at scale. The approach is modular, explainable, and readily integrable with existing tooling.

11. Future Work

Potential future enhancements include:

- Incorporating transformer-based models for code analysis,
- Adding contextual awareness through linked ticket history and team activity,
- Allowing for reviewer feedback to fine-tune model predictions,

- Extending to other domains like documentation quality or test coverage alignment.

With the rise of DevOps and continuous delivery, intelligent automation like this will be essential to sustaining velocity without compromising on reliability or traceability.

12. System Workflow and Component Reports

To better understand the orchestration of this research framework, this section outlines the step-by-step workflow and describes the responsibilities of each module or script in the pipeline. This comprehensive view helps to visualize how the pieces fit together to achieve semantic alignment and risk classification.

12.1 Overview of the Workflow

The system architecture is composed of five main stages:

1. **Synthetic Data Generation**
2. **SBERT-Based Semantic Similarity Scoring**
3. **Feature Engineering and Label Generation**
4. **Training and Evaluation using Random Forest**
5. **Prediction, Classification, and Output Export**

Each stage is handled by a dedicated script with specific responsibilities.

12.2 Enhanced Data Generation (enhanced_data_generation.py)

This script simulates a realistic software engineering environment by creating 500 JIRA tickets and 500 pull requests. Each record is generated using randomly selected templates that mimic authentic engineering tasks such as new features, bug fixes, or performance improvements.

Key Functions:

- Uses templates to create diverse ticket/PR descriptions.
- Randomizes fields such as task type, technologies, and modules.
- Calculates domain-specific metadata: lines added, deleted, comment count, file paths changed, and risk metrics.
- Introduces 10% misalignment artificially to support the training of semantic detection models.
- Saves datasets into jira_tickets.csv and pull_requests.csv.

12.3 Semantic Similarity Analysis (sbert.py)

This module applies a pre-trained Sentence-BERT model (all-MiniLM-L6-v2) to evaluate how well each PR description aligns with the corresponding JIRA ticket.

Key Workflow:

- Loads the tickets and PRs from CSV files.
- Merges them based on matching ticket IDs.
- Encodes each description using SBERT into vector embeddings.
- Computes cosine similarity for each pair.
- Flags low-scoring pairs (below threshold of 0.7).
- Outputs a new file alignment_scores.csv with semantic scores and selected PR metadata.

This component provides the semantic alignment score that is later used as one of the core features in the machine learning model.

12.4 Random Forest Risk Prediction (optimized_random_forest.py)

This script is the analytical core of the system, integrating SBERT scores with engineered metadata features and training a Random Forest classifier to predict the risk associated with each pull request.

Core Responsibilities:

- Merges pull_requests.csv with alignment_scores.csv on PR IDs.
- Parses JSON fields such as changed file paths.
- Computes derived features:
    - Critical file count
    - Lines modified
    - PR age
    - Author experience
    - Code churn ratio
- Generates risk labels based on heuristic rules.
- Trains a Random Forest classifier on the data.
- Evaluates using metrics (accuracy, precision, recall, F1-score).
- Outputs feature importance for interpretability.
- Saves predictions into pr_predictions.csv with confidence probabilities.

12.5 Output and Decision Support

The final output is a comprehensive dataset (pr_predictions.csv) where each pull request has:

- A binary risk label (High Risk or Low Risk)
- A risk probability score
- A full feature breakdown used during prediction

This report can be used in dashboards or CI/CD pipelines to:

- Prioritize reviews for high-risk or misaligned PRs.
- Generate alerts or recommendations for developers.
- Track the distribution of semantic alignment across teams and time periods.

12.6 Summary of Interaction Between Components

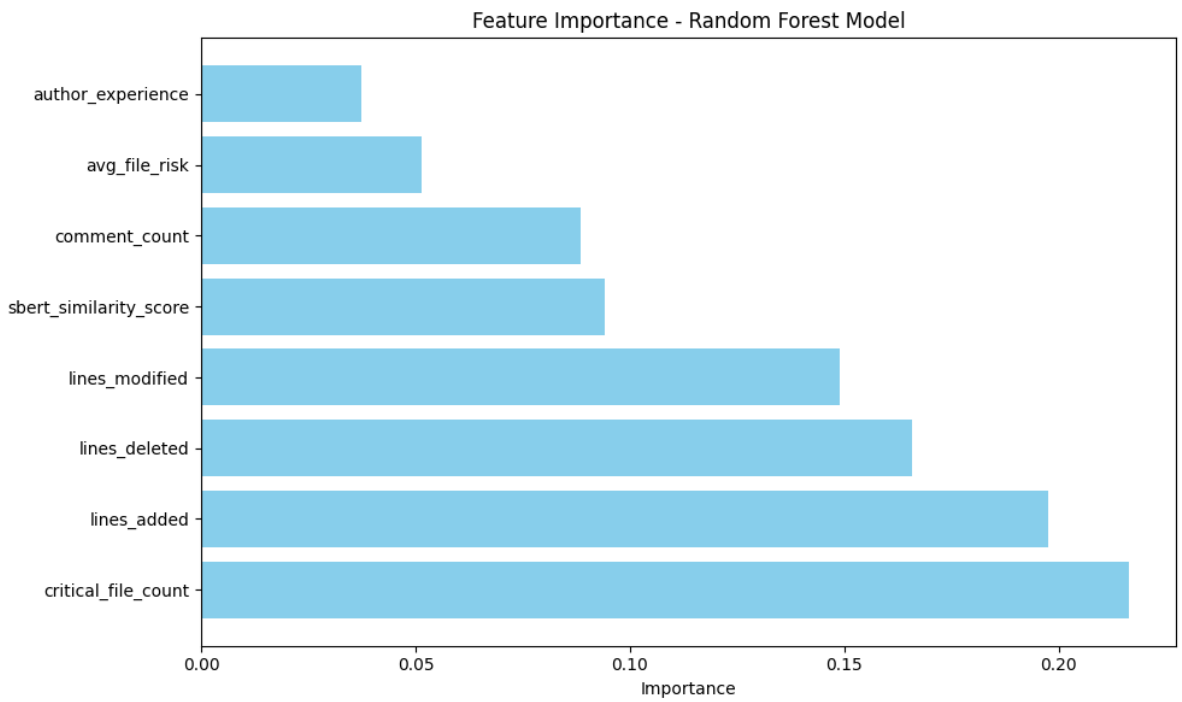
Component	Input	Output	Role in Workflow
enhanced_data_generation.py	None (random seed/templates)	jira_tickets.csv, pull_requests.csv	Generates synthetic issue and PR records
sbert.py	Tickets and PRs CSVs	alignment_scores.csv	Computes semantic alignment using SBERT
optimized_random_forest.py	PRs and Alignment scores	pr_predictions.csv	Predicts high-risk PRs using engineered features
SBERT Model	Text pairs	Similarity score	Provides semantic understanding of descriptions
Random Forest Model	Feature vectors	Binary risk label, probability	Learns risk pattern and makes predictions

---

13. Concluding Summary

Through this structured and extensible architecture, the system ensures that both semantic correctness and engineering risk are assessed early in the development lifecycle. Each component builds on the outputs of its predecessor, forming a pipeline that is modular, interpretable, and practical for integration in modern software development workflows.

This combination of natural language understanding with structured decision-making exemplifies the potential of AI in augmenting software quality assurance, enabling scalable and intelligent review processes across organizations.


Feature Importance - Random Forest Model
author_experience
avg_file_risk
comment_count
sbert_similarity_score
lines_modified
lines_deleted
lines_added
critical_file_count
0.00
0.05
0.10
0.15
0.20
Importance